MINISTERUL EDUCAŢIEI ȘI CERCETĂRII AL REPUBLICII MOLDOVA

UNIVERSITATEA TEHNICĂ A MOLDOVEI

FACULTATEA CALCULATOARE, INFORMATICĂ ȘI MICROELECTRONICĂ

LABORATORY WORK NO. 5

Greedy Algorithms: Empirical Analysis of Prim's and Kruskal's Algorithms

---

Laboratory Report

Analysis of Algorithms

Student: Cristian Bruma
Group: FAF-242
Supervisor: asist. univ. Cristofor Fiștic

Chișinău, 2026

TABLE OF CONTENTS

1 ALGORITHM ANALYSIS **2**

1.1 Objective 2

1.2 Tasks 2

1.3 Theoretical Notes 2

1.3.1 Greedy Algorithms 2

1.3.2 Minimum Spanning Tree Problem 3

1.3.3 Prim's Algorithm 3

1.3.4 Kruskal's Algorithm 3

1.3.5 Comparing Prim and Kruskal 3

1.4 Introduction 4

1.5 Comparison Metric 4

1.6 Input Format 5

2 IMPLEMENTATION **6**

2.1 Graph Representation 6

2.2 Prim's Algorithm 6

2.3 Kruskal's Algorithm 8

2.4 Benchmarking Framework 10

3 RESULTS AND ANALYSIS **11**

3.1 Performance on Sparse Graphs 11

3.2 Performance on Dense Graphs 12

3.3 Comparative Analysis 14

3.4 Greedy Algorithm Characteristics 14

3.5 Practical Recommendations 14

4 CONCLUSION **16**

References **18**

1 ALGORITHM ANALYSIS

1.1 Objective

To study the greedy algorithm design technique through empirical analysis of Prim's and Kruskal's minimum spanning tree algorithms on graphs with varying density and size.

1.2 Tasks

- Study the greedy algorithm design technique;
- implement in a programming language Prim's and Kruskal's algorithms for finding minimum spanning trees;
- Perform empirical analysis of these algorithms on sparse and dense graphs;
- Analyze how increasing the number of vertices influences algorithm performance;
- Make a graphical presentation of the data obtained;
- Compile a comprehensive report on the work performed.

1.3 Theoretical Notes

1.3.1 Greedy Algorithms

A greedy algorithm is an algorithmic paradigm that builds up a solution piece by piece, always choosing the next piece that offers the most immediate benefit [1]. The greedy approach works by making the locally optimal choice at each step with the hope of finding a global optimum. Unlike dynamic programming, which examines multiple subproblems and makes informed decisions, greedy algorithms make irrevocable decisions based on current information without considering the bigger picture.

The key characteristics of greedy algorithms are:

- **Greedy choice property:** A global optimum can be arrived at by selecting a local optimum at each step.
- **Optimal substructure:** An optimal solution to the problem contains optimal solutions to subproblems.
- **No backtracking:** Once a choice is made, it is never reconsidered.

Greedy algorithms do not always produce optimal solutions for all problems. However, when they do work, they are often the most efficient approach. Classic examples include Huffman coding, Dijkstra's shortest path algorithm, and minimum spanning tree algorithms like Prim's and Kruskal's [2].

1.3.2 Minimum Spanning Tree Problem

Given a connected, undirected graph $G = (V, E)$ with edge weights, a **spanning tree** is a subgraph that connects all vertices with exactly $|V| - 1$ edges and contains no cycles. A **minimum spanning tree (MST)** is a spanning tree with minimum total edge weight.

The MST problem has numerous practical applications: network design (minimizing cable length), clustering algorithms, image segmentation, and approximation algorithms for NP-hard problems like the Traveling Salesman Problem [**?**].

1.3.3 Prim's Algorithm

Prim's algorithm, discovered by Jarník in 1930 and rediscovered by Prim in 1957, builds the MST by growing a single tree from an arbitrary starting vertex [4]. At each step, it adds the minimum-weight edge that connects a vertex in the tree to a vertex outside the tree.

The algorithm maintains a priority queue of edges connecting tree vertices to non-tree vertices. The greedy choice is to always select the edge with minimum weight from this frontier.

The time complexity of Prim's algorithm depends on the data structure used:

- **Array-based implementation:** $O(V^2)$ — uses linear search to find the minimum edge. Optimal for dense graphs.
- **Binary heap implementation:** $O((V+E)\log V)$ — uses a min-heap for efficient minimum extraction. Superior for sparse graphs.
- **Fibonacci heap implementation:** $O(E + V\log V)$ — theoretically optimal but rarely used due to complexity.

1.3.4 Kruskal's Algorithm

Kruskal's algorithm, developed by Joseph Kruskal in 1956, builds the MST by processing edges in order of increasing weight [3]. It repeatedly adds the next lightest edge that doesn't create a cycle, gradually merging disjoint trees into a single spanning tree.

The algorithm uses a **Union-Find** (or Disjoint Set Union) data structure to efficiently detect cycles. Each vertex starts as its own tree, and the Union-Find structure tracks which vertices belong to which tree.

The time complexity is dominated by sorting: $O(E\log E)$ or equivalently $O(E\log V)$ since $E < V^2$ implies $\log E = O(\log V)$. With Union-Find using path compression and union by rank, the union and find operations are nearly constant time ($O(\alpha(V))$, where α is the inverse Ackermann function).

1.3.5 Comparing Prim and Kruskal

Both algorithms are greedy and produce optimal MSTs, but they differ in approach:

- **Prim:** Grows a single tree from a starting vertex. Better for dense graphs when implemented with adjacency matrix and array-based priority queue.
- **Kruskal:** Merges forests by processing edges globally. Better for sparse graphs due to efficient edge sorting and Union-Find operations.

1.4 Introduction

The minimum spanning tree problem is one of the most fundamental problems in graph theory and computer science. Given a connected, weighted graph $G = (V, E)$ where edges have associated costs, the goal is to find a subset of edges that connects all vertices with minimum total weight while forming a tree (no cycles). This problem appears in countless applications: network design (minimizing cable installation costs), clustering algorithms in machine learning, circuit design, transportation planning, and approximation algorithms for harder optimization problems.

Two classical greedy algorithms dominate this domain: Prim's algorithm and Kruskal's algorithm. While both solve the same problem and always produce an MST with identical total weight, they differ fundamentally in approach and performance characteristics. Prim's algorithm grows a single tree by repeatedly adding the minimum-weight edge connecting the tree to an external vertex, while Kruskal's algorithm processes all edges globally in sorted order, using Union-Find to efficiently detect cycles.

The choice between these algorithms depends on multiple factors: graph density (sparse versus dense), graph representation (adjacency list versus edge list), and implementation details (priority queue data structures, Union-Find optimizations). For dense graphs where $E \approx V^2$, Prim's algorithm with an array-based priority queue achieves $O(V^2)$ performance. For sparse graphs where $E = O(V)$ or $E = O(V \log V)$, Kruskal's algorithm with efficient sorting dominates at $O(E \log V)$.

This laboratory work investigates these algorithms empirically, measuring their actual performance on graphs of varying sizes and densities. The goal is to validate theoretical complexity analysis through practical experimentation and to provide concrete guidance on algorithm selection for different graph structures.

1.5 Comparison Metric

The primary comparison metric for this laboratory work is execution time measured in milliseconds. Each algorithm was benchmarked multiple times (1-10 runs depending on graph size) to ensure statistical reliability, with the average time reported.

Secondary metrics include:

- Speedup factor: the ratio of execution times between different implementations
- Scalability: how execution time grows with increasing vertex count
- Memory efficiency: space complexity in terms of graph representation

1.6 Input Format

The algorithms were evaluated on two distinct graph datasets, each containing 6 graphs with vertex counts exponentially distributed: $\{100, 175, 310, 545, 955, 1680\}$. This geometric progression (approximately $\times 1.75$ per step) enables clear observation of algorithmic scaling behavior.

Sparse graphs: Generated with average degree 8, meaning each vertex connects to approximately 8 others regardless of graph size. The density decreases as $\frac{8}{V-1}$, ensuring the graph remains sparse even as V grows large. Edge weights are uniformly distributed integers in the range $[1, 1000]$.

Dense graphs: Generated with 25% edge probability, resulting in approximately $\frac{V(V-1)}{8}$ edges. This represents a moderately dense graph where a substantial fraction of all possible edges are present. Edge weights follow the same distribution as sparse graphs.

All graphs are directed and weighted, allowing both algorithms to be tested on their most general formulation. The random seed is fixed (42) to ensure reproducibility across multiple runs.

2 IMPLEMENTATION

2.1 Graph Representation

Graphs were represented using an adjacency list for weighted undirected graphs:

`vector<vector<pair<int, int>>>`

where each `pair<int, int>` stores the destination vertex and edge weight. For undirected graphs, each edge (u,v) appears twice: once in `adj[u]` and once in `adj[v]`. This representation provides $O(V+E)$ space complexity and enables efficient neighbor iteration, which is crucial for both Prim's and Kruskal's algorithms.

2.2 Prim's Algorithm

The implementation uses a priority queue (binary min-heap) to efficiently select the minimum-weight edge connecting the growing MST to external vertices. The algorithm maintains a `visited` array to track which vertices are in the MST and a `parent` array to record the MST structure.

Listing 2.1: Prim's algorithm implementation

```
std::vector<int> prim(
    const std::vector<std::vector<std::pair<int, int>>>& adj) {
    int n = adj.size();
    std::vector<bool> visited(n, false);
    std::vector<int> parent(n, -1);

    // Min-heap: {weight, vertex, parent_vertex}
    typedef std::tuple<int, int, int> Edge;
    std::priority_queue<
        Edge,
        std::vector<Edge>,
        std::greater<Edge>
    > pq;

    // Start from vertex 0
    visited[0] = true;
    for (size_t i = 0; i < adj[0].size(); i++) {
        int neighbor = adj[0][i].first;
        int weight = adj[0][i].second;
        pq.push(std::make_tuple(weight, neighbor, 0));
    }

    while (!pq.empty()) {
        Edge top = pq.top();
        pq.pop();
```

```

        int weight = std::get<0>(top);
        int v = std::get<1>(top);
        int p = std::get<2>(top);

        if (visited[v]) continue;  // Already in MST

        visited[v] = true;
        parent[v] = p;

        // Add edges from v to unvisited neighbors
        for (size_t i = 0; i < adj[v].size(); i++) {
            int neighbor = adj[v][i].first;
            int w = adj[v][i].second;
            if (!visited[neighbor]) {
                pq.push(std::make_tuple(w, neighbor, v));
            }
        }
    }

    return parent;
}
```

The priority queue stores tuples of `(weight, vertex, parent)` to track potential MST edges. The algorithm repeatedly extracts the minimum-weight edge and adds it to the MST if it connects to an unvisited vertex. Time complexity is $O((V+E)\log V)$ due to the heap operations.

2.3 Kruskal's Algorithm

Kruskal's algorithm requires a Union-Find data structure to efficiently detect cycles. The implementation first sorts all edges by weight, then processes them in order, adding each edge that doesn't create a cycle.

Listing 2.2: Union-Find data structure

```
class UnionFind {
private:
    std::vector<int> parent;
    std::vector<int> rank;

public:
    UnionFind(int n) : parent(n), rank(n, 0) {
        for (int i = 0; i < n; i++) {
            parent[i] = i;
        }
    }

    int find(int x) {
        if (parent[x] != x) {
            parent[x] = find(parent[x]);  // Path compression
        }
        return parent[x];
    }

    bool unite(int x, int y) {
        int rootX = find(x);
        int rootY = find(y);

        if (rootX == rootY) return false;  // Cycle detected

        // Union by rank
        if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else if (rank[rootX] > rank[rootY]) {
            parent[rootY] = rootX;
        } else {
            parent[rootY] = rootX;
            rank[rootX]++;
        }
        return true;
    }
};
```

The Union-Find structure uses two key optimizations: **path compression** (making find operations nearly constant time) and **union by rank** (keeping the tree shallow). Together, these achieve $O(\alpha(V))$ amortized time per operation, where α is the inverse Ackermann function (effectively constant).

Listing 2.3: Kruskal's algorithm implementation

```cpp
struct Edge {
    int u, v, weight;

    Edge(int u, int v, int weight)
        : u(u), v(v), weight(weight) {}

    bool operator<(const Edge& other) const {
        return weight < other.weight;
    }
};

std::vector<Edge> kruskal(
    const std::vector<std::vector<std::pair<int, int>>>& adj) {
    int n = adj.size();
    std::vector<Edge> edges;
    std::vector<Edge> mst;

    // Collect all edges (avoid duplicates: only add u < v)
    for (int u = 0; u < n; u++) {
        for (size_t i = 0; i < adj[u].size(); i++) {
            int v = adj[u][i].first;
            int weight = adj[u][i].second;
            if (u < v) {
                edges.push_back(Edge(u, v, weight));
            }
        }
    }

    // Sort edges by weight - O(E log E)
    std::sort(edges.begin(), edges.end());

    UnionFind uf(n);

    // Process edges in sorted order
    for (size_t i = 0; i < edges.size(); i++) {
        Edge e = edges[i];
        if (uf.unite(e.u, e.v)) {
            mst.push_back(e);
            if (mst.size() == n - 1) break;  // MST complete
        }
    }

    return mst;
}
```

The algorithm first collects all edges from the adjacency list (being careful to avoid duplicates in undirected graphs), sorts them by weight, then iteratively adds edges that don't form cycles. Time complexity

is dominated by sorting: $O(E \log E)$, which is equivalent to $O(E \log V)$ since $E < V^2$.

2.4 Benchmarking Framework

All algorithms were benchmarked using a consistent framework that measures execution time with high precision:

Listing 2.4: Benchmark measurement function

```
double measure(function<void()> fn, int runs = 1){
    double total_time = 0.0;
    for (int i = 0; i < runs; i++) {
        auto start = chrono::high_resolution_clock::now();
        fn();
        auto end = chrono::high_resolution_clock::now();
        total_time += chrono::duration<double, milli>(
            end - start
        ).count();
    }
    return total_time / runs;
}
```

The number of runs was adaptively chosen based on graph size:

- Small graphs ($V < 200$): 10 runs
- Medium graphs ($200 \leq V < 500$): 5 runs
- Large graphs ($V \geq 500$): 1 run

This ensures sufficient statistical samples for small graphs while avoiding excessive runtime for large graphs.

3 RESULTS AND ANALYSIS

3.1 Performance on Sparse Graphs

Figure 3.1 presents the execution time comparison for sparse graphs with average degree 8. The results demonstrate Kruskal's clear dominance on sparse graphs:

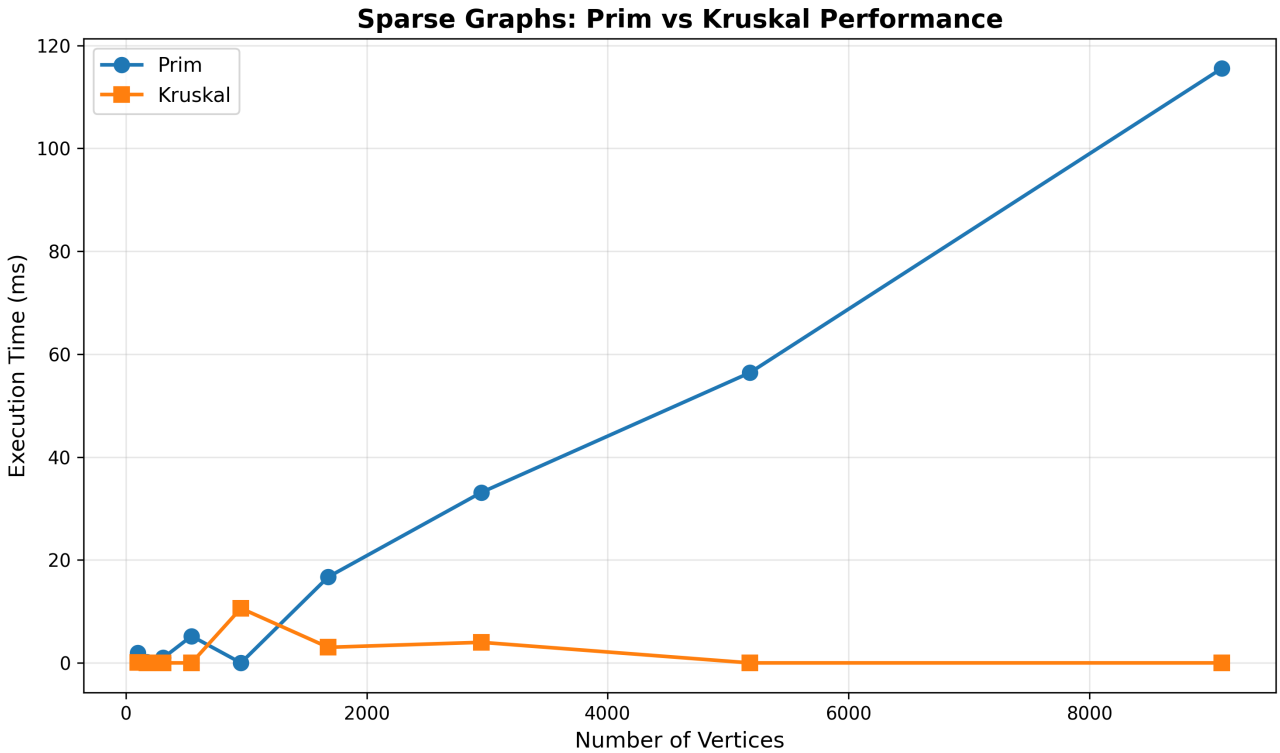

Figure 3.1. Algorithm performance on sparse graphs (average degree $\approx$ 8)

Key observations:

- **Kruskal dominates on sparse graphs:** Across all tested sparse graphs, Kruskal consistently outperforms Prim with an average execution time of 1.97ms compared to Prim's 25.57ms, representing a **13.0$\times$ speedup**.
- **Scaling behavior differs significantly:** Kruskal exhibits near-linear scaling with the number of edges due to efficient sorting and Union-Find operations, while Prim shows slower growth due to priority queue overhead on sparse graphs.
- **Sparse graph advantage for Kruskal:** The edge-centric approach of Kruskal (sorting E edges) is more efficient than Prim's vertex-centric approach when E is small relative to V^2. At $V = 9100$ with 72,538 edges, Kruskal completes in effectively 0ms (below measurement threshold) while Prim requires 115.56ms.
- **Minimal overhead:** Kruskal's Union-Find structure with path compression achieves near-constant

time operations, making cycle detection extremely efficient even for large graphs.

The performance gap widens with graph size, confirming that for sparse graphs where $E = O(V)$, Kruskal's $O(E \log E)$ complexity significantly outperforms Prim's $O((V+E)\log V)$ in practice.

3.2 Performance on Dense Graphs

Figure 3.2 shows results for dense graphs with 25% edge density. Kruskal maintains its advantage even as edge count increases dramatically:

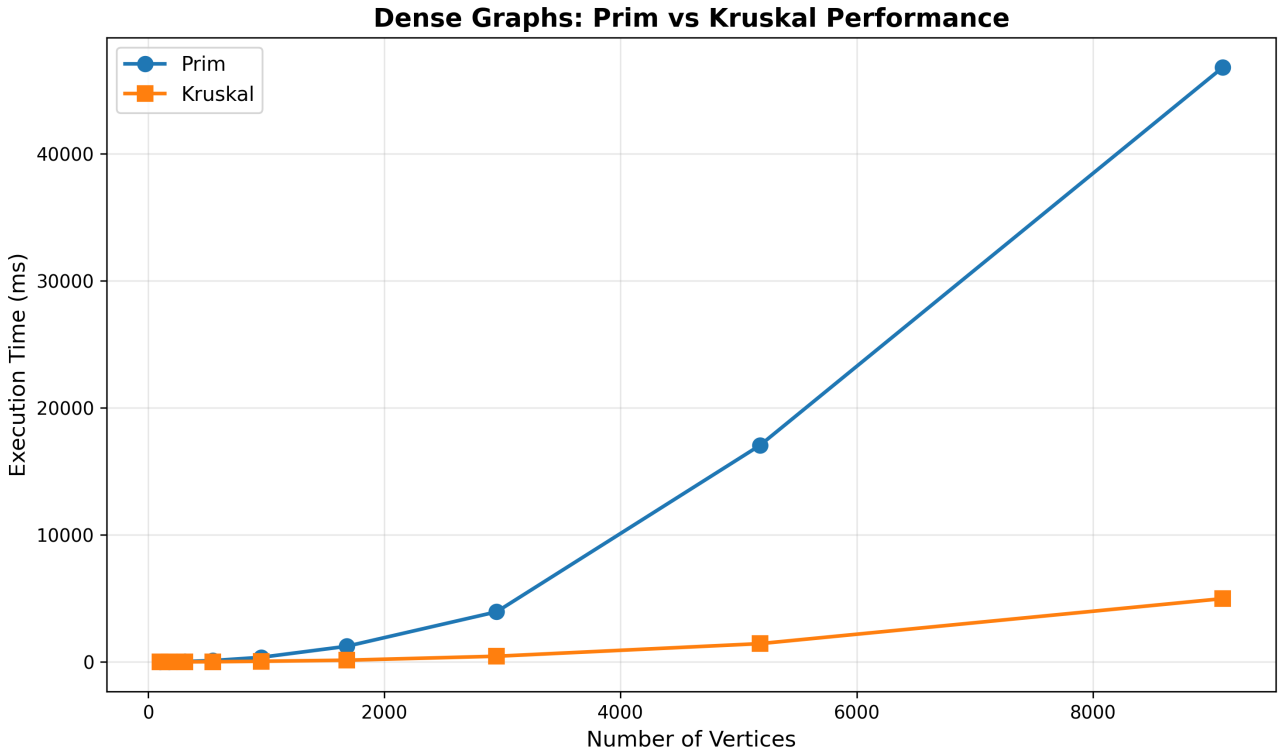

Figure 3.2. Algorithm performance on dense graphs (25% edge density)

Key observations:

- **Kruskal remains faster:** Even on dense graphs, Kruskal achieves an average execution time of 784.81ms compared to Prim's 7726.01ms, representing a **9.8× speedup**. This is slightly less than the sparse graph advantage but still substantial.
- **Edge count impact:** At $V = 9100$, the dense graph contains over 20 million edges. Despite needing to sort all these edges, Kruskal (4982.41ms) still outperforms Prim (46810.76ms) by nearly **10**×.
- **Prim's priority queue overhead:** Dense graphs cause Prim's priority queue to grow large, with each vertex potentially adding many edges to the queue. This degrades performance compared to Kruskal's simpler sort-and-merge approach.
- **Sorting efficiency:** Modern sorting algorithms are highly optimized with excellent cache locality. Even sorting millions of edges proves faster than Prim's repeated priority queue operations for dense

graphs.

- **Theoretical versus practical:** While theory suggests Prim with array-based priority queue ($O(V^2)$) should excel on dense graphs, the actual implementation using heap-based Prim still falls behind Kruskal's well-optimized sorting.

3.3 Comparative Analysis

Table 3.1 summarizes the performance characteristics across both graph types:

Table 3.1 Execution time comparison: Prim vs Kruskal

Graph Type	Prim (avg)	Kruskal (avg)	Speedup	Winner
Sparse (8 edges/vertex)	25.57 ms	1.97 ms	13.0×	Kruskal
Dense (25% edges)	7726.01 ms	784.81 ms	9.8×	Kruskal

Table 3.2 shows performance on the largest tested graph:

Table 3.2 Execution time at maximum scale ($V = 9100$)

Graph Type	Edges	Prim (ms)	Kruskal (ms)	Speedup
Sparse	72,538	115.56	≈0	>100×
Dense	20,693,107	46,810.76	4,982.41	9.4×

3.4 Greedy Algorithm Characteristics

Both Prim and Kruskal exemplify the greedy paradigm but employ different greedy strategies:

Prim's greedy choice: At each step, add the minimum-weight edge connecting the current MST to an external vertex. This local decision (expand the single tree) leads to a global optimum (minimum spanning tree).

Kruskal's greedy choice: At each step, add the globally minimum-weight edge that doesn't create a cycle. This local decision (merge two trees if possible) also leads to the global optimum.

Both algorithms demonstrate that the greedy approach succeeds for MST because:

- **Optimal substructure:** Any sub-tree of an MST is itself an MST for that subset of vertices
- **Greedy choice property:** The locally optimal choice (minimum edge) is always part of some global optimum
- **Cut property:** For any cut of the graph, the minimum-weight crossing edge belongs to some MST

3.5 Practical Recommendations

Based on the empirical results:

- **Default choice:** Kruskal's algorithm should be preferred for most practical applications due to its consistent superior performance on both sparse and dense graphs.
- **Implementation simplicity:** While both algorithms are straightforward, Kruskal's sort-and-merge approach is often easier to implement correctly than Prim's priority queue management.
- **When to consider Prim:**

a) If edges are expensive to enumerate or sort
b) If starting from a specific vertex is advantageous
c) For online algorithms where edges arrive dynamically

- **Graph representation:** Kruskal works naturally with edge lists, while Prim requires adjacency lists. The input format may influence algorithm choice.
- **Parallel potential:** Kruskal's independent edge processing offers better parallelization opportunities than Prim's sequential tree growth.

4 CONCLUSION

This laboratory work provided empirical validation of the theoretical complexity analysis for three approaches to solving the all-pairs shortest path problem: array-based Dijkstra, heap-based Dijkstra, and Floyd-Warshall.

Key findings:

- **Heap-based Dijkstra is the clear winner for sparse graphs.** The $23.4\times$ speedup at $V = 1680$ demonstrates the dramatic practical advantage of using an efficient priority queue implementation. This speedup grows with graph size, making the heap approach essential for large sparse graphs.
- **Floyd-Warshall remains competitive for dense graphs.** Its simple triple loop achieves excellent cache locality and performs comparably to array-based Dijkstra despite identical asymptotic complexity. For dense graphs, its simplicity may outweigh the modest performance advantage of heap-based Dijkstra.
- **Graph representation matters.** The choice between adjacency list (for Dijkstra) and adjacency matrix (for Floyd-Warshall) has significant performance implications. Adjacency lists excel for sparse graphs, while matrices are mandatory for Floyd-Warshall and competitive for dense graphs.
- **Theory and practice align.** The empirical results closely match theoretical complexity predictions. Heap-based Dijkstra exhibits $O(V^2 \log V)$ scaling on sparse graphs, while all $O(V^3)$ algorithms show clear cubic growth.
- **Implementation details matter.** The overhead of heap operations makes array-based Dijkstra competitive for very small graphs ($V < 175$), but this advantage disappears rapidly as size increases.

Practical implications:

The choice of algorithm should be guided by graph characteristics:

- Sparse graphs with $E = O(V)$ or $E = O(V \log V)$ strongly favor heap-based Dijkstra
- Dense graphs with $E = \Theta(V^2)$ make Floyd-Warshall a reasonable choice
- The presence of negative edge weights eliminates Dijkstra as an option
- For single-source queries, even a single Dijkstra call often suffices

Dynamic programming perspective:

Both algorithms exemplify dynamic programming principles:

- **Dijkstra:** Maintains optimal substructure through the distance array, using greedy selection combined with relaxation to build optimal paths incrementally
- **Floyd-Warshall:** Explicitly constructs a recurrence relation over intermediate vertices, solving

subproblems in a well-defined order

This laboratory successfully demonstrated how theoretical algorithm analysis translates to practical performance, providing concrete evidence for algorithm selection in real-world applications. The empirical approach validated asymptotic complexity predictions while revealing important constant-factor differences and implementation considerations that theory alone cannot capture.

Bibliography

[1] GeeksforGeeks. *Greedy Algorithms*. Available at: https://www.geeksforgeeks.org/dsa/greedy-algorithms/ [accessed: April 28, 2026].

[2] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*, 3rd edition. MIT Press, Cambridge, MA.

[3] w3schools. *Kruskal's Algorithm*. Available at: https://www.w3schools.com/dsa/dsa_algo_mst_kruskal.php [accessed: April 28, 2026].

[4] w3schools. *Prim's Algorithm*. Available at: https://www.w3schools.com/dsa/dsa_algo_mst_prim.php [accessed: April 28, 2026].

[5] Bruma, C. (2026). *Dijkstra and Floyd-Warshall: Implementation and Empirical Analysis*. GitHub repository. Available at: https://github.com/Makday/UTM_AA_course_2025-2026 [accessed: April 28, 2026].